

lion tokens, achieving a better performance (lower perplexity) than the original StreamLLM method [52] across various cache size. Further comparisons are reported in Appendix C.4.

Q2: Does the number of shots during inference effects the effectiveness of H₂O? A2: Effective across zero-shot to ten-shots inference. We further examine H₂O under different numbers of shots during inference, and the results are reported in Table 10 and Figure 8. With different shots inference, our H₂O achieves matching performance (difference less than 1.00%) as the full model across different downstream tasks. The "Local" strategy encounters significant performance degradation (up to 37.00%). Such results demonstrate the effectiveness of our H₂O under different inference scenarios. More details about zero-shot and one-shot inference are reported in Appendix C.3.

Q3: Compatible with Quantization? A3: Yes. To pursue further efficiency, we show the compatibility of H₂O with another orthogonal approach, *i.e.*, quantization in Table 6. We use OPT-30B as our base model and COPA, OpenBookWA, and PiQA as evaluation tasks. Intuitively sparsity and quantization are highly related so combining them might introduce larger errors. Surprisingly the combination almost always achieves better accuracy than H₂O or quantization alone. Experiments about throughput improvement are detailed in Appendix C.2.

Q4: When does H₂O match the baseline with full KV embeddings? A4: With both H₂ and the recent tokens. We investigate the separate effects of KV embeddings of H₂ and the local tokens. We conduct experiments on 4 tasks with OPT-13B and OPT-30B. For each task, we compare the performance of three KV cache eviction policies, including only the KV embeddings of H₂, only the ones of local tokens, and our H₂O that keep both. As shown in Table 9, only retaining the embeddings of H₂ or local tokens can't maintain a similar performance as the model using full embeddings, with a performance degradation from 2.85% to 22.75%. Incorporating both components, our H₂O successfully retains the baseline performance with full embeddings. Besides, the model with only H₂ shows a consistent improvement against the one with only local tokens, which indicates H₂ might contribute more to maintaining the performance.

Q5: Extra benefits from H₂O? A5: Increased diversity of generated text. Besides all the benefits of our H₂O, we also observe an bonus introduced by H₂O, *i.e.*, the improved diversity of generated content. The results are reported in Appendix C.1. Given the same prompts, we visualize the generated text of the models with different KV cache budgets. Compared with the model of full KV cache, our H₂O can generate sentences with fewer repeated words and more creativity.

6 Conclusion and Discussion

In this paper, we study one of the key bottlenecks of LLM deployment, KV cache, particularly for long-content and large-batch generation applications. We propose H₂O, a simple KV cache eviction policy for significantly reducing its memory footprint. The main insight of our approach is the recognition of a subset of tokens, known as Heavy Hitters, which contribute the most value when computing attention scores. We formulate the KV cache eviction as a dynamic submodular problem and provide the theoretical guarantees for our algorithm. Through extensive evaluations, we demonstrate that H₂O can significantly improve end-to-end throughput and decrease latency in wall-clock time, without compromising the generation quality of LLMs across a variety of tasks.

7 Acknowledgement

Ying Sheng and Clark Barrett are partly supported by NSF-2110397 and the Stanford Center for Automated Reasoning. Z. Wang is in part supported by a Google Research Scholar Award and the NSF AI Institute for Foundations of Machine Learning (IFML).

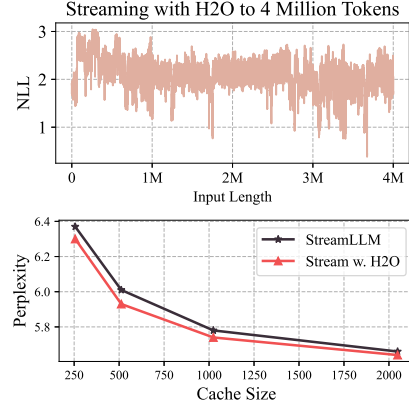


Figure 5: (Upper) streaming with H₂O to handle inputs with sequence lengths of four million tokens. (Bottom) Perplexity comparison between the original StreamLLM method and our H₂O, results are collected on the first text sample of PG-19 [54].

References

- [1] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.
- [2] Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. Wordcraft: story writing with large language models. In *27th International Conference on Intelligent User Interfaces*, pages 841–852, 2022.
- [3] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [4] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. Benchmarking large language models for news summarization. *arXiv preprint arXiv:2301.13848*, 2023.
- [5] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *arXiv preprint arXiv:2211.05102*, 2022.
- [6] Laszlo A Belady, Robert A Nelson, and Gerald S Shedler. An anomaly in space-time characteristics of certain programs running in a paging machine. *Communications of the ACM*, 12(6):349–353, 1969.
- [7] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [9] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [10] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.
- [11] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [12] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [13] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- [14] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. *arXiv preprint arXiv:2304.08467*, 2023.
- [15] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation. In *Zenodo*. <https://doi.org/10.5281/zenodo.5371628>, September 2021.
- [16] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- [17] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale. *arXiv preprint arXiv:2207.00032*, 2022.

- [18] HuggingFace. Hugging face accelerate. <https://huggingface.co/docs/accelerate/index>.
- [19] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E Gonzalez, et al. High-throughput generative inference of large language models with a single gpu. *arXiv preprint arXiv:2303.06865*, 2023.
- [20] Elias Frantar and Dan Alistarh. Massive language models can be accurately pruned in one-shot. *arXiv preprint arXiv:2301.00774*, 2023.
- [21] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.
- [22] Lu Yin, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Mykola Pechenizkiy, Yi Liang, Zhangyang Wang, and Shiwei Liu. Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity. *arXiv preprint arXiv:2310.05175*, 2023.
- [23] Elias Frantar, Saleh Ashkboos, Torsten Hoeffler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [24] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.
- [25] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *arXiv preprint arXiv:2206.01861*, 2022.
- [26] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.
- [27] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.
- [28] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.
- [29] Joshua Ainslie, Tao Lei, Michiel de Jong, Santiago Ontañón, Siddhartha Brahma, Yury Zemlyanskiy, David Uthus, Mandy Guo, James Lee-Thorp, Yi Tay, et al. Colt5: Faster long-range transformers with conditional computation. *arXiv preprint arXiv:2303.09752*, 2023.
- [30] Sotiris Anagnostidis, Dario Pavlo, Luca Biggio, Lorenzo Noci, Aurelien Lucchi, and Thomas Hoffmann. Dynamic context pruning for efficient and interpretable autoregressive transformers. *arXiv preprint arXiv:2305.15805*, 2023.
- [31] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.
- [32] Hanrui Wang, Zhekai Zhang, and Song Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 97–110. IEEE, 2021.
- [33] Elizabeth J O’neil, Patrick E O’neil, and Gerhard Weikum. The lru-k page replacement algorithm for database disk buffering. *Acm Sigmod Record*, 22(2):297–306, 1993.
- [34] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. Lrfu: A spectrum of policies that subsumes the least recently used and least frequently used policies. *IEEE transactions on Computers*, 50(12):1352–1361, 2001.
- [35] Valerii Likhoshesterov, Krzysztof Choromanski, and Adrian Weller. On the expressive power of self-attention matrices. *arXiv preprint arXiv:2106.03764*, 2021.
- [36] Benjamin L Edelman, Surbhi Goel, Sham Kakade, and Cyril Zhang. Inductive biases and variable creation in self-attention mechanisms. In *International Conference on Machine Learning*, pages 5793–5831. PMLR, 2022.
- [37] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.